

Compiler Optimizations for QAOA

(Invited Paper)

Yuchen Zhu*
Rensselaer Polytechnic Institute
Troy, NY, USA

Yidong Zhou*
Rensselaer Polytechnic Institute
Troy, NY, USA

Jinglei Cheng*
Purdue University
West Lafayette, IN, USA

Yuwei Jin*
Rutgers University
New Brunswick, NJ, USA

Boxi Li
Forschungszentrum Jülich
Jülich, Germany

Siyuan Niu
Lawrence Berkeley National
Laboratory
Berkeley, CA, USA

Zhiding Liang
Rensselaer Polytechnic Institute
Troy, NY, USA

ABSTRACT

The Quantum Approximate Optimization Algorithm (QAOA) is one of the most promising candidates for achieving quantum advantage over classical computers. However, existing compilers lack specialized methods for optimizing QAOA circuits. There are circuit patterns inside the QAOA circuits, and current quantum hardware has specific qubit connectivity topologies. Therefore, we propose Coqa to optimize QAOA circuit compilation tailored to different types of quantum hardware. Our method integrates a linear nearest-neighbor (LNN) topology and efficiently map the patterns of QAOA circuits to the LNN topology by heuristically checking the interaction based on the weight of problem Hamiltonian. This approach allows us to reduce the number of SWAP gates during compilation, which directly impacts the circuit depth and overall fidelity of the quantum computation. By leveraging the inherent patterns in QAOA circuits, our approach achieves more efficient compilation compared to general-purpose compilers. With our proposed method, we are able to achieve an average of 30% reduction in gate count and a 39x acceleration in compilation time across our benchmarks.

1 INTRODUCTION

Quantum computing is an emerging technology with the potential to tackle problems that are currently beyond the reach of classical computation, such as tasks in cryptography [31], quantum chemistry [4, 34, 23], and optimization [21, 22]. Quantum computers' ability to process complex computations in parallel, leveraging quantum phenomena like superposition and entanglement, offers

great potentials for industries ranging from drug discovery to secure communications. However, different types of noise in quantum hardware, which can cause quantum states to collapse and lead to computational errors, presents significant challenges, hindering the realization of practical quantum applications. Overcoming these obstacles is essential for unlocking the full potential of quantum computing and making it a viable tool for solving real-world problems.

Quantum error correction (QEC) has been proposed as a solution to mitigate noise and enable fault-tolerant quantum computing (FTQC). However, the substantial overhead required—often involving millions of qubits—renders it impractical for the near-term devices we have today. Currently, we are in the noisy intermediate-scale quantum (NISQ) era [28], characterized by quantum systems with a limited number of qubits of constrained quality. In this context, variational quantum algorithms (VQAs) [27, 5, 35, 24, 15] have emerged as a more practical approach. These hybrid quantum-classical algorithms leverage the strengths of both quantum and classical computing, requiring fewer qubits and shallower circuit depths. They are specifically designed to exploit the potential of NISQ devices, aiming to achieve quantum advantages in the near term.

The Quantum Approximate Optimization Algorithm (QAOA) [7, 39, 38] stands out as one of the most promising algorithms within the realm of VQAs for addressing combinatorial optimization problems, such as Max-Cut [9] and Low Autocorrelation Binary Sequence (LABS) [29]. It leverages the power of quantum mechanics to explore complex solution spaces more efficiently than classical algorithms. QAOA works by preparing a parameterized quantum circuit that encodes the problem, and then iteratively refining the parameters through a classical-quantum optimization loop to minimize a classical cost function tailored to the specific problem at hand. This hybrid approach allows QAOA to potentially find high-quality solutions more quickly than traditional methods, making it a key contender for demonstrating quantum advantage in practical applications.

When evaluating QAOA on quantum hardware, it is essential to compile the quantum circuits to the specific architecture of the hardware. For example, superconducting quantum hardware typically has nearest-neighbor connectivity constraints, which create

*These authors contributed to the work equally.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1077-3/24/10

<https://doi.org/10.1145/3676536.3697127>

challenges in executing quantum circuits as intended. A critical step of this compilation is known as the qubit mapping problem, which involves aligning the quantum circuit with the hardware’s constraints. This process includes two key steps: first, determining the initial layout by mapping the circuit qubits onto the hardware’s physical qubits, and second, inserting SWAP gates to ensure that two-qubit gates meet the hardware’s topology requirements. Our paper focuses on optimizing this mapping process within the compiler for QAOA circuits, aiming to enhance the performance and efficiency of quantum computations on current hardware.

When optimizing the mapping process for quantum circuits, key objectives often include reducing the number of SWAP gates, especially since these gates typically have higher error rates—enhancing the overall circuit fidelity, and minimizing circuit depth to improve execution efficiency. Achieving these goals is crucial for maintaining the integrity of quantum computations, particularly in the noisy environment of current quantum hardware. While general-purpose compilers like Qiskit [3], TKet [32], and BQSKit [36] are designed to handle a wide range of quantum circuits, QAOA circuits possess unique properties that call for specialized compilers to achieve optimal performance.

One of the distinct characteristics of QAOA circuits is that their two-qubit gates are commutative, meaning the order in which these gates are applied can be adjusted without affecting the final outcome. This property provides additional flexibility in re-ordering the gates during the optimization process. Specialized QAOA compilers can take advantage of this flexibility, allowing for more efficient circuit layouts and reducing the number of required SWAP gates [16, 2, 1]. Moreover, unlike other algorithms that might benefit from the specific connectivity features of IBM’s heavy hex hardware, QAOA circuits do not necessarily require such complex connectivity. Instead, a linear transformation of the topology map can be employed to achieve lower compilation overhead, accelerating the process and further enhancing circuit efficiency. By leveraging these approaches, our QAOA-specific compiler can significantly improve the performance and reliability of quantum computations.

Our contributions are as follows:

- We are the first to explore the LNN topology for QAOA circuits.
- We analyze the structure of QAOA circuits and the LNN topology, and propose a method to efficiently map QAOA circuits onto the LNN topology.
- We evaluate our method on IBM’s heavy-hex topology and demonstrate a 30% reduction in gate count and a 39x acceleration in compilation time.

The rest of the paper is structured as follows: In Section 2, we introduce the background of QAOA and LNN. In Section 3, we present the motivation for our work. Section 4 discusses the technical details of our method, and Section 5 demonstrates the results on IBM quantum computers. We cover related work in Section 6 and conclude in Section 7.

2 BACKGROUND

In this session, we explore the fundamental concepts associated with QAOA and the challenges of its implementation on quantum hardware. The primary challenges involve the compilation and

optimization of quantum circuits to align with the constraints of modern quantum architectures.

2.1 Introduction to QAOA

QAOA is a hybrid quantum-classical algorithm widely used to find approximate solutions to combinatorial optimization problems. More specifically, QAOA involves five main steps: problem setup, unitary transformation, ansatz construction, classical optimization, and measurement.

- (1) **Problem Setup:** Encode the optimization problem into a cost Hamiltonian H_C , with the goal of finding its ground state.
- (2) **Unitary Transformations:** Define the unitary operations:

$$U(H_C, \gamma) = e^{-i\gamma H_C}, \quad U(H_B, \beta) = e^{-i\beta H_B}$$

where H_C is the cost Hamiltonian, H_B is a mixing Hamiltonian, and γ and β are parameters to be optimized.

- (3) **Ansatz Construction:** Create the ansatz state $|\psi(\gamma, \beta)\rangle$ by applying a series of unitary transformations starting from an initial state $|\psi_0\rangle$:

$$|\psi(\gamma, \beta)\rangle = U(H_B, \beta_p)U(H_C, \gamma_p) \dots U(H_B, \beta_1)U(H_C, \gamma_1)|\psi_0\rangle$$

- (4) **Classical Optimization:** Optimize the parameters γ and β using classical algorithms to minimize the expectation value of H_C :

$$\langle H_C \rangle = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle$$

- (5) **Measurement:** Measure the quantum state $|\psi(\gamma, \beta)\rangle$ to obtain a classical bit string, representing an approximate solution to the optimization problem.

2.2 Compilation of QAOA

QAOA involves creating a quantum circuit with alternating layers of unitary transformations and problem-specific Hamiltonian interactions. The challenge in compiling QAOA circuits stems from their specific structure and the need to align them with hardware constraints.

2.2.1 Problem-Graph Induced Circuit. The structure of a QAOA circuit is dictated by the input problem graph. For instance, in the QAOA-Maxcut problem, qubits correspond to vertices in the graph, and controlled-phase (CPHASE) gates correspond to edges. This unique feature of QAOA circuits is their flexibility in gate ordering due to the commutativity of two-qubit gates. As illustrated in Fig. 1, multiple valid gate arrangements can be applied to the same problem graph, allowing for optimization in circuit compilation.

2.2.2 Compilation Objectives. QAOA circuits are characterized by their use of parameterized unitary gates, which are optimized through classical feedback loops. The compilation process must address the following parts:

- **Qubit Mapping and Hardware Constraints:** A major challenge in compiling QAOA circuits is efficiently mapping logical qubits to physical qubits on a quantum chip, which involves addressing connectivity constraints. Due to limited qubit connectivity and the potential for gate errors, logical qubits often need to be connected using SWAP gates. This mapping must also account for hardware limitations such as

gate fidelities and qubit coherence times to ensure accurate computation.

- **Gates Optimization:** Optimizing the order of unitary gates in QAOA circuits is crucial for minimizing circuit depth and error rates. The commutativity of two-qubit gates in QAOA provides flexibility in gate sequencing, which can be leveraged to enhance performance.

By addressing these aspects—problem-graph structure, qubit mapping, and gate optimization—the compilation of QAOA circuits aims to enhance performance and adapt to the constraints of quantum hardware effectively. Fig. 2 illustrates how different optimization strategies can affect circuit depth and gate count, highlighting the impact of gate ordering and SWAP insertion on overall circuit efficiency.

3 MOTIVATION

Limitation in Quantum Hardwares. Quantum devices are inherently error-prone, and the variability in error rates across different qubits and gates can have a significant impact on the performance of quantum circuits. In a superconducting device, such as IBM ibm_quebec [11], the median Error per Clifford (ECR) error rate is $6.931e-3$, a crucial metric for understanding the fidelity of quantum operations, particularly when implementing error correction protocols. The median T1 and T2 time are 284.52 us and 214.31 us respectively. These two metrics indicate how long a qubit can retain its quantum state before it loses energy. It also has other errors, such as state preparation and measurement (SPAM) errors, crosstalk errors, etc.

These errors make the execution of quantum programs lose fidelity. The impact of these various errors is exacerbated in deep circuits, where the errors accumulate over many operations and execution time, leading to substantial deviations from the expected outcomes. As a result, minimizing circuit depth and gate count is not just a matter of efficiency but also essential for reducing the overall error rate and enhancing the reliability of quantum computations.

In addition to the errors in a quantum device, the qubit connection tends to be sparse to avoid crosstalk errors [10, 30, 6]. For example, Fig. 3 represents the qubit connection in the ibm_quebec device with readout error in each qubit and ECR error in each connection. Each circle represents a qubit and a link represents where a two-qubit gate could be executed. The color indicates the error variability. A lighter color indicates a higher error rate.

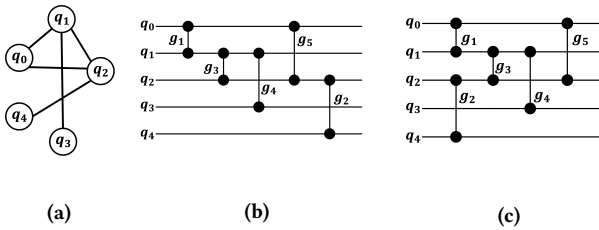


Figure 1: A five-qubit QAOA-Maxcut circuit example: (a) the input problem graph. (b) and (c) are two valid circuits.

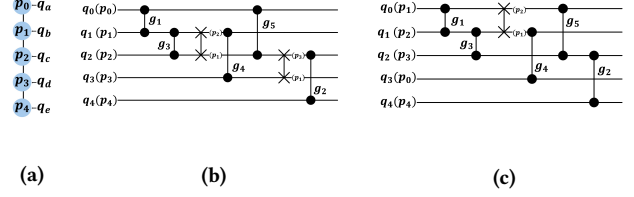


Figure 2: (a) Hardware coupling map; (b) One possible compiled version of the circuit corresponding to the input problem graph, (c) Another possible compiled version of the same circuit. Due to the qubit mapping process, the number of gates and the depth of the circuit in (c) are reduced compared to (b), illustrating a more efficient compilation.

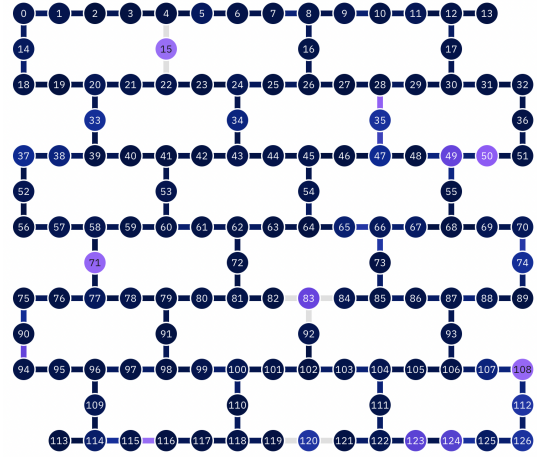


Figure 3: Qubit Connection in IBM_Quebec.

Such sparse connectivity requires more SWAP gate insertion in the circuit compilation. Those additional SWAP gates increase the gate count, extend the circuit execution, and exacerbate the negative impact of errors in the program execution outcome.

Global view in the QAOA compilation. Qubit mapping is a NP-hard Problem [37, 18, 26]. The existing generic quantum compilers either leverage SAT solvers [25, 33], which suffer from scalability issues, or use heuristic solutions that do not guarantee the quality of the compilation results [19]. The challenge becomes even more significant in the compilation of the QAOA problem, where gate commutativity can be exploited. Although, some domain-specific compilers for QAOA program are proposed [16, 2, 1], they still face significant drawbacks, such as scalability issues [33, 25] and unexpectedly high compilation costs [17, 20, 2].

However, as the pattern introduced in [12, 8] for quantum fourier transformation (QFT) circuit compilation in Heavy-hex architecture, we found that there exists a solution with global consideration for the compilation of QAOA. QFT circuit has all-to-all interactions with certain dependency constraints. The QAOA circuit for the clique input problem graph also has all-to-all interactions. We could impose the same gate dependency constraint in QFT to QAOA

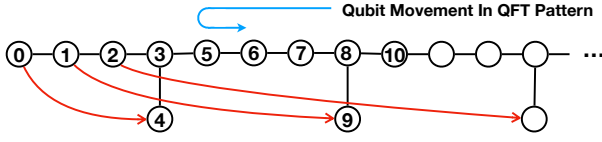


Figure 4: Expanded Heavy-hex Architecture and QFT Pattern.

circuit and reuse the QFT pattern to compile QAOA program. Although this gate dependency is unnecessary to QAOA, it gives us an idea of what the upper bound of the compilation cost of QAOA program could be.

The idea of the QFT pattern in [12, 8] is stretched in Fig. 4. Firstly, it ignores some connections and expands the heavy-hex architecture into a linear nearest-neighbor connected line with certain dangling points. Then, all qubits move to the left and turn around once they hit the boundary. When the qubit with the smallest index moves to the position above the dangling point, it switches with the qubit at the dangling point. For example, qubit 1 will reside at the first dangling point. This pattern gives an upper bound in circuit depth with $5n + O(1)$, where n is the total number of qubits.

Even though, [14] also introduced a pattern for compiling QAOA in Heavy-hex architecture, the solution consisted of two iterations of linear nearest neighboring (LNN) pattern with some extra interactions between off-path qubits and interactions between path qubits and off-path qubits. However, the upper bound for those extra operations is unclear.

Adaptation from QFT pattern to generic QAOA. There are two types of adaptations we have to do. Adaptation 1: Resolving unnecessary dependencies in QAOA compilation. This adaptation is straightforward. In the QAOA compilation using the QFT pattern, we can directly execute all gates where the corresponding qubits are mapped to connected physical qubits. Some gates in QAOA may be executed earlier than their scheduled execution in the QFT pattern. When we encounter these gates later, we can ignore the operations that have already been scheduled.

Adaptation 2, resolving redundant SWAP gates for sparse input problem graphs. As we discussed above, we can use QFT pattern for compiling the clique problem graph in QAOA. Any graph is the sub-graph of the clique graph. We can simply reuse QFT pattern for a random graph as long as we do not execute gates that the corresponding edges do not appear in the input problem graph. However, the number of SWAP gates and where they are performed is fixed. It is possible that only a few of remaining gates are scheduled in the late stage of the pattern and we have to go through all SWAP gates that are pre-defined in the pattern. Those redundant SWAP gates could be avoided heuristically. For more details about the adaptation, please refer to our next section.

4 METHODOLOGY

4.1 Modified Topology for Repetitive Patterns

To take advantages of the topology information during circuit compilation, it is crucial to identify repetitive patterns. However, these

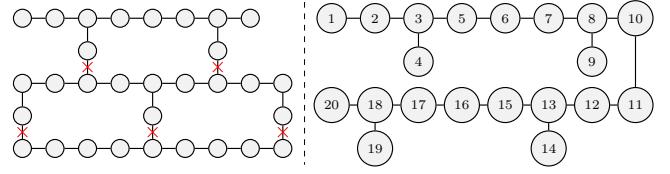


Figure 5: Modified coupling graph of heavy-hex architecture to generate repetitive patterns.

patterns can be challenging to recognize directly within a heavy-hex architecture. Different from linear nearest neighbour (LNN) and 2D grid, heavy-hex architecture can not be described as the duplication of a group. Therefore we process the coupling graph of heavy-hex is through removing some qubit pairs. As shown in Fig 5, after removing certain qubit pairs, the final coupling graph resembles the linear structure. After the modification, we now have a repetitive pattern that is composed of four qubits in a line and a qubit dangled. Also, the qubits' labels in the modified architecture also differ from the original heavy-hex architecture where the label is consecutive in the same line. As shown in the right side of Fig 5, the dangling qubit is labeled after the adjacent line qubit. This labeling method ensures that previous qubits do not need relabeled when more repetitive patterns are concatenated to the modified architecture.

This modification could offer several advantages. Two-qubit entanglement operations require a frequency offset from neighboring qubit transition frequencies to avoid collisions. The reduced connectivity in the new architecture results in fewer frequency constraints for two-qubit interactions, potentially enhancing fidelity. More importantly, the architecture finds a balanced point between hardware connectivity and software requirements. The coupling graph exhibits a clear repetitive pattern while maintaining most connections. Additionally, based on the heavy-hex architecture, multiple approaches can be employed to transition to the new architecture. For instance, for each dangling qubit, either the upper or lower connection can be removed, providing a broad design space for further optimizations.

4.2 Linear Nearest Neighbor

The linear Nearest neighbor (LNN) architecture is a fundamental model in quantum computing, where qubits are connected in a linear sequence, allowing direct interaction only between adjacent qubits. This architecture is intuitive and resembles the topologies of many current quantum computers, where qubit connections are sparse and the number of connections grows linearly with the number of qubits. The linear arrangement in LNN simplifies the overall topology, reducing its complexity and enabling the acceleration of the mapping process with a guaranteed upper bound on the circuit size. This makes it easier to implement quantum algorithms efficiently on LNN architectures, as opposed to relying on general-purpose compilers. Recent works [12] have explored the LNN with Quantum Fourier transform (QFT) algorithm, and we are the first to explore this method on QAOA algorithm. Unlike QFT, where all-to-all qubit connectivity is often beneficial to perform the necessary quantum gates efficiently, QAOA does not necessitate a

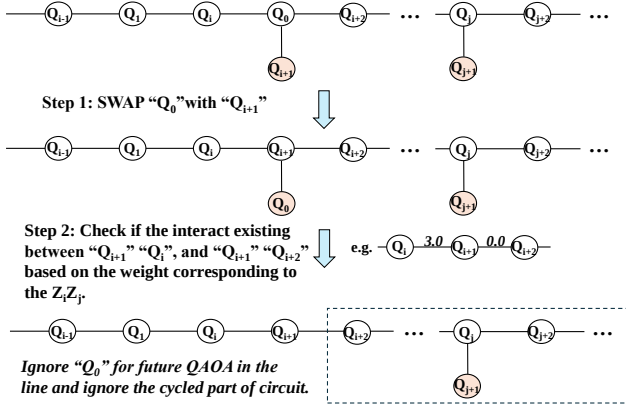


Figure 6: Recursive pattern: reducing one dangling node from the coupling graph after two steps: one SWAP gate between the qubit above and the dangling point itself, and then check the interaction between pair-formed nearby qubits based on the weight corresponding to the $Z_i Z_j$.

fully connected qubit architecture, so we need to carefully remove redundant circuit during the compilation. we will introduce details in the following subsections.

4.3 Linear Structure with Dangling Point

In a heavy-hex lattice, dangling points are qubits that are connected to the main body of qubits but only have one direct neighbor. These qubits are usually evenly distributed on the edge or corner of the qubit lattice, as indicated in Fig. 3. Since dangling points are qubits with limited connectivity, we need to pay special attention to these qubits when we design and implement the mapping process. For example, we should consider to place qubits with less connections onto the dangling points to minimize the need of SWAP gates. We can also use dangling points to store qubits that are not immediately needed but will be used later in the quantum program. The first time qubit q_0 is positioned above (connected to) q_{i+1} (the dangling qubit), as illustrated in Figure 6, we execute a SWAP operation on q_0 and q_{i+1} . This results in the qubits along the main line taking a configuration similar to an intermediate step in the linear QFT process [13, 8], where the line begins with q_1 and ends at q_{N-1} , just before q_1 swaps with q_{i+1} . The process then continues with the standard steps of performing QAOA from q_1 to q_{N-1} along the straight line until completion. Importantly, whenever a qubit q_j (where $j > i$) is moved adjacent to q_0 in the dangling position for the first time, we pause the algorithm and the algorithm concludes. According to the properties of QAOA, after performing the SWAP operation between q_0 and q_{i+1} , we need to check the interaction based on the weights in the problem Hamiltonian. For example, in Figure 6, if the weight between q_{i+1} and q_1 is 3.0, and the weight between q_{i+1} and q_{i+2} is 0.0, then there is no interaction between q_{i+1} and q_{i+2} . In this case, we can safely ignore the redundant circuits following q_{i+2} .

4.4 General Complexity Analysis

Given our initial labeling, all previous qubits' labels remain unchanged after adding more repetitive groups of qubits and most operations will overlap. Therefore, the time complexity of our algorithms could be inducted through the calculation of extra steps required to add a qubit group. Considering a group of 5 more qubits are added to a finished mapping of $5n$ qubits, the extra operations are composed of two phases: moving q_n back towards the final dangling point and moving q_{5n+4} forward towards the first position. It takes 10 steps for moving q_n to the dangling points and an extra 15 steps to move q_{5n+4} to the correct place. Therefore, for every 5 qubits, at most 25 steps are required to complete the QAOA mapping, and the time complexity of our mapping algorithm is $5N + O(1)$, ensuring a linear upbound for the mapping process.

5 EVALUATION

5.1 Experimental Setup

The experiments in this study were conducted on a laptop equipped with the Apple M1 Pro chip. This system features a 10-core CPU, consisting of 8 performance cores and 2 efficiency cores, supporting 10 concurrent threads with a base clock frequency of 3.2 GHz. The device also includes 16 GB of unified memory. We utilized Qiskit 1.1.0 as the software platform for our experiments. Our results are compared against QAIM (Qubit Allocation and Initial Mapping) [2] and the structured method proposed by Jin et al. [16]. To create QAOA instances of different graphs, We use the library Networkx in python to generate Erdős-Rényi random graphs and random k -regular graphs. For each type of graphs and each vertex number, we vary graph density to 0.3 and 0.5.

5.2 Compilation Time

The Table 1 compares the compilation times of various QAOA compilers including QAIM, a Structured Method, and our proposed methods across different qubit counts. The results demonstrate that our method consistently outperforms the other methods, showing significantly lower compilation times across all tested qubit counts. For instance, at 65 qubits, our method takes only 0.012 seconds, compared to 292 seconds for QAIM and 0.8 seconds for the Structured Method. As the qubit count increases, the efficiency of our method remains obvious with a minimal increase in time even for 1025 qubits where it takes 8.68 seconds, while QAIM exceeds 5 hours. In summary, our method demonstrates superior performance in terms of compilation time.

5.3 Results for Small-Scale Circuits

The results in Table 2 compare the performance of three different methods: our method, the structured method, and QAIM. The experiments are conducted with QAOA circuits on different graph instances, evaluating both circuit depth and gate count. These instances are relatively small, and our method achieves significantly shorter compilation times. For these small-scale circuits, our method generally demonstrates comparable circuit depths and gate counts. For example, in the 25-0.3 graph, our method produces 202 gates, compared to 194 for the structured method and 215 for QAIM. The

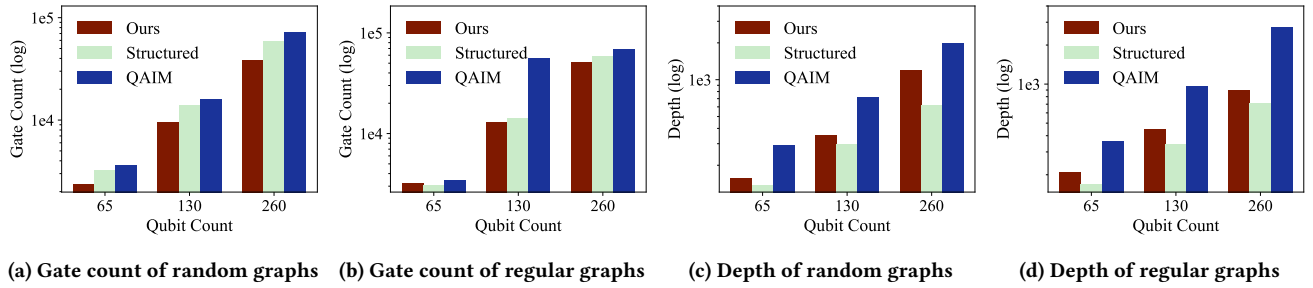


Figure 7: Comparison of large-scale QAOA circuit compilation results including random graphs and regular graphs.

overall trend indicates that our method offers competitive performance with compilation times that are only a fraction of those required by other methods.

5.4 Results for Large-Scale Circuits

The graphs in Figure 7 present a comparative analysis of gate count and depth for random and regular graphs with varying qubit counts (65, 130, and 260) using three methods: our method, the structured method, and QAIM. The gate count, as shown on a logarithmic scale in Figure 7a and Figure 7b, indicates that our method consistently produces the lowest number of gates across all qubit counts for both random and regular graphs. This demonstrates that our method is more efficient in terms of gate count. For circuit depth, as presented in Figure 7c and Figure 7d, our method consistently outperforms QAIM, showing lower depths for all qubit counts. At the same time, our method’s circuit depth is slightly higher and comparable to that of the structured method. Overall, our method is capable of producing the lowest gate counts and competitive circuit depths with very fast compilation times.

Table 1: Compilation time comparison of various QAOA compilers.

Qubit count	Time(s)		
	QAIM	Structured Method	Ours
65	292	0.8	0.012
125	4215	6.4	0.029
515	> 5hrs	47.4	1.14
1025	> 5hrs	337	8.68

6 RELATED WORK

Previous work [16] introduces a method for optimizing QAOA circuits, addressing the limitations of existing compilers in terms of performance and scalability when dealing with large-scale quantum circuits. This structured compilation method leverages the regularity of quantum hardware to ensure linear depth on multi-dimensional architectures, significantly outperforming existing approaches in both performance and scalability. [2] present four novel methodologies - QAIM (Qubit Allocation and Initial Mapping), IP (Instruction Parallelization), IC (Incremental Compilation), and VIC

Table 2: Comparison of small-scale QAOA circuit compilation results where graphs are composed of random graphs and regular graphs, e.g., "10-0.3" represents a random graph with 10 nodes and an edge generation probability of 0.3, and "20-6" represents a regular graph with 20 nodes and each node has 6 edges.

Graphs	Depth			Gate Count		
	Ours	Structured	QAIM	Ours	Structured	QAIM
10-0.3	12	6	11	19	19	23
15-0.3	26	15	25	59	64	65
20-0.3	33	22	33	95	119	129
25-0.3	35	29	52	202	194	215
10-0.5	24	15	22	47	44	49
15-0.5	29	17	32	89	87	95
20-0.5	36	24	53	167	159	190
25-0.5	39	34	66	271	259	307
20-6	28	17	28	109	107	113
20-8	35	22	36	150	142	155
25-10	45	30	71	247	235	279

(Variation-aware Incremental Compilation) — offer significant improvements over existing approaches to optimize the compilation of QAOA circuits, addressing challenges that arise with NISQ devices. And [1] builds on efforts by introducing scalable, compiler-agnostic methodologies that achieve significant reductions in both gate count and circuit depth, improving the overall performance of QAOA circuits. Recent work [12, 8] proposes circuit mapping solutions tailored for modern multi-dimensional quantum architectures designed for heavy-hex, significantly improving both efficiency and scalability over previous approaches, however, this work is specialized for linear-depth QFT.

7 CONCLUSION

In this paper, we propose to take advantages of the patterns in QAOA circuits and the topology of existing quantum computers to optimize the compilation process. Specifically, we unroll the topology of current quantum computers to a linear nearest neighbor (LNN) configuration and implement a tailored mapping algorithm for QAOA circuits which are determined by the graph instances they represent. As a result, we achieve a 30% reduction in gate count and a 39x acceleration in the compilation process.

REFERENCES

- [1] Mahabubul Alam, Abdullah Ash-Saki, and Swaroop Ghosh. 2020. An efficient circuit compilation flow for quantum approximate optimization algorithm. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [2] Mahabubul Alam, Abdullah Ash-Saki, and Swaroop Ghosh. 2020. Circuit compilation methodologies for quantum approximate optimization algorithm. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 215–228.
- [3] Gadi Aleksandrowicz et al. 2019. Qiskit: an open-source framework for quantum computing. (2019).
- [4] Yudong Cao et al. 2019. Quantum chemistry in the age of quantum computing. *Chemical reviews*, 119, 19, 10856–10915.
- [5] Jinglei Cheng, Hanrui Wang, Zhiding Liang, Yiyu Shi, Song Han, and Xuehai Qian. 2022. Topgen: topology-aware bottom-up generator for variational quantum circuits. arXiv preprint, arXiv:2210.08190. (2022).
- [6] Yongshan Ding, Pranav Gokhale, Sophia Fuhui Lin, Richard Rines, Thomas Propson, and Frederic T. Chong. 2020. Systematic crosstalk mitigation for superconducting qubits via frequency-aware compilation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, (Oct. 2020). doi: 10.1109/micro50266.2020.00028.
- [7] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*.
- [8] Xiangyu Gao, Yuwei Jin, Minghao Guo, Henry Chen, and Eddy Z Zhang. 2024. Linear depth qft over ibm heavy-hex architecture. *arXiv preprint arXiv:2402.09705*.
- [9] Gian Giacomo Guerreschi and Anne Y. Matsuura. 2019. Qaoa for max-cut requires hundreds of qubits for quantum speed-up. *Scientific reports*, 9, 1, 1–7.
- [10] Fei Hua et al. 2023. A synergistic compilation workflow for tackling crosstalk in quantum machines. (2023). <https://arxiv.org/abs/2207.05751> arXiv: 2207.05751 [quant-ph].
- [11] IBM Quantum. 2024. Ibm quantum resources. https://quantum.ibm.com/services/resources?system=ibm_quebec. Accessed: 2024-07-22. (2024).
- [12] Yuwei Jin, Xiangyu Gao, Minghao Guo, Henry Chen, Fei Hua, Chi Zhang, and Eddy Z Zhang. 2023. Quantum fourier transformation circuits compilation. *arXiv preprint arXiv:2312.16114*.
- [13] Yuwei Jin, Fei Hua, Yanhao Chen, Ari Hayes, Chi Zhang, and Eddy Z Zhang. 2023. Exploiting the regular structure of modern quantum architectures for compiling and optimizing programs with permutable operators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 108–124.
- [14] Yuwei Jin, Fei Hua, Yanhao Chen, Ari Hayes, Chi Zhang, and Eddy Z. Zhang. 2024. Exploiting the regular structure of modern quantum architectures for compiling and optimizing programs with permutable operators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4 (ASPLOS '23)*. Association for Computing Machinery, Vancouver, BC, Canada, 108–124. ISBN: 9798400703942. doi: 10.1145/3623278.3624751.
- [15] Yuwei Jin, Zirui Li, Fei Hua, Tianyi Hao, Huiyang Zhou, Yipeng Huang, and Eddy Z. Zhang. 2024. Tetris: a compilation framework for vqa applications in quantum computing. (2024). <https://arxiv.org/abs/2309.01905> arXiv: 2309.01905 [quant-ph].
- [16] Yuwei Jin, Jason Luo, Lucent Fong, Yanhao Chen, Ari B Hayes, Chi Zhang, Fei Hua, and Eddy Z Zhang. 2021. A structured method for compilation of qaoa circuits in quantum computing. *arXiv preprint arXiv:2112.06143*.
- [17] Lingling Lao and Dan E Browne. 2022. 2qan: a quantum compiler for 2-local qubit hamiltonian simulation algorithms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 351–365.
- [18] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the qubit mapping problem for nisq-era quantum devices. (2019). <https://arxiv.org/abs/1809.02573> arXiv: 1809.02573 [cs.ET].
- [19] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the qubit mapping problem for nisq-era quantum devices. In *ASPLOS*, 1001–1014.
- [20] Gushu Li, Anbang Wu, Yunong Shi, Ali Javadi-Abhari, Yufei Ding, and Yuan Xie. 2022. Paulihedral: a generalized block-wise compiler optimization framework for quantum simulation kernels. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 554–569.
- [21] Yangyang Li, Mengzhuo Tian, Guangyuan Liu, Cheng Peng, and Licheng Jiao. 2020. Quantum optimization and quantum learning: a survey. *Ieee Access*, 8, 23568–23593.
- [22] Zhiding Liang et al. 2023. Hybrid gate-pulse model for variational quantum algorithms. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [23] Zhiding Liang et al. 2024. Napa: intermediate-level variational native-pulse ansatz for variational quantum algorithms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*.
- [24] Zhiding Liang et al. 2022. Variational quantum pulse learning. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 556–565.
- [25] Abtin Molavi, Amanda Xu, Martin Diges, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2022. Qubit mapping and routing via maxsat. In *2022 55th IEEE/ACM international symposium on Microarchitecture (MICRO)*. IEEE, 1078–1091.
- [26] Siyuan Niu, Adrien Suau, Gabriel Staffelbach, and Aida Todri-Sanial. 2020. A hardware-aware heuristic for the qubit mapping problem in the nisq era. *IEEE Transactions on Quantum Engineering*, 1, 1–14.
- [27] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O’Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. In *Nature Communications* number 1. Vol. 5, 4213. ISBN: 2041-1723. doi: 10.1145/237814.237866.
- [28] John Preskill. 2018. Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79.
- [29] Ruslan Shaydulin et al. 2024. Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem. *Science Advances*, 10, 22, eadm6761.
- [30] Sarah Sheldon, Easwar Magesan, Jerry M. Chow, and Jay M. Gambetta. 2016. Procedure for systematically tuning up cross-talk in the cross-resonance gate. *Physical Review A*, 93, 6, (June 2016). doi: 10.1103/physreva.93.060302.
- [31] Peter W. Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41, 2, 303–332.
- [32] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. TKET: A Retargetable Compiler for NISQ devices. *Quantum Science and Technology*, 6, (Nov. 2020). doi: 10.1088/2058-9565/ab8e92.
- [33] Daniel Bochen Tan, Murphy Yuezheng Niu, and Craig Gidney. 2024. A sat scalpel for lattice surgery: representation and synthesis of subroutines for surface-code fault-tolerant quantum computing. *arXiv preprint arXiv:2404.18369*.
- [34] Hanrui Wang, Yongshan Ding, Jiaqi Gu, Yujun Lin, David Z. Pan, Frederic T. Chong, and Song Han. 2022. Quantumnas: noise-adaptive search for robust quantum circuits. In *HPCA2022*. IEEE. IEEE.
- [35] Hanrui Wang, Zirui Li, Jiaqi Gu, Yongshan Ding, David Z. Pan, and Song Han. 2022. Qoc: quantum on-chip training with parameter shift and gradient pruning. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. IEEE, San Francisco, USA, 655–660.
- [36] Ed Younis, Costin C Iancu, Wim Lavrijsen, Marc Davis, Ethan Smith, and USDOE. 2021. Berkeley quantum synthesis toolkit (bqskit) v1. (Apr. 2021). doi: 10.11578/dc.20210603.2.
- [37] Chi Zhang, Ari B. Hayes, Longfei Qiu, Yuwei Jin, Yanhao Chen, and Eddy Z. Zhang. 2021. Time-optimal qubit mapping. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. Association for Computing Machinery, Virtual, USA, 360–374. ISBN: 9781450383172. doi: 10.1145/3445814.3446706.
- [38] Zewen Zhang, Roger Paredes, Bhuvanesh Sundar, David Quiroga, Anastasios Kyriillidis, Leonardo Duenas-Osorio, Guido Pagano, and Kaden R. A. Hazzard. 2024. Grover-qaoa for 3-sat: quadratic speedup, fair-sampling, and parameter clustering. (2024). <https://arxiv.org/abs/2402.02585> arXiv: 2402.02585 [quant-ph].
- [39] Yingyue Zhu, Zewen Zhang, Bhuvanesh Sundar, Alaina M Green, C Huerta Alderete, Nhung H Nguyen, Kaden R A Hazzard, and Norbert M Linke. 2022. Multi-round qaoa and advanced mixers on a trapped-ion quantum computer. *Quantum Science and Technology*, 8, 1, (Nov. 2022), 015007. doi: 10.1088/2058-9565/ac91ef.